Project Proposal: Persistent Sandboxes for Long-Running Coding Agents

How we got here

1. Coding agents run code they write, which can be wrong or harmful, so each agent runs in its own isolated sandbox, usually rented from Daytona or E2B.
2. Agent tasks run for hours or days, and agents often sit idle waiting on a person or another agent. Paying for idle compute is waste, so idle sandboxes get stopped.
3. Stopping a sandbox clears its memory, so running processes end and work in progress is lost. Saving memory along with disk solves it.
4. Doing this automatically, for many sandboxes, takes a control plane: something that decides when to pause, saves memory and disk together, and restores sandboxes without conflicts.

Who experiences it

- **For:** small teams or solo developers who self-host coding agents in full dev-environment sandboxes and want idle compute reclaimed without interrupting work in progress.
- **Not for:**
  - **eval/benchmark platforms.** Each trial must start clean and be thrown away, so persisting state only adds storage cost and risks leaking one trial into the next.
  - **long running** apps that are always serving and never idle (full-stack app).
  - **short code-execution jobs.** Fast startup matters more than resuming.

Engineering problem: state persistence

Problem: When a long-running coding agent's sandbox is paused to save compute, what must survive, and how do we save and restore it automatically so the agent picks up with its work intact?

What state lives in a sandbox

- **Disk state:** files such as the repo, installed packages, build output, logs and PID files.
- **Memory state:** running processes, open ports and connections, in-memory variables, and terminal sessions.

The tradeoff: continuity vs. snapshot cost

	Save disk only	**Save disk + memory**
Examples	Daytona containers, Netclode	Daytona VMs, E2B
Pros	Small, fast pauses; cheaper storage; simpler to scale	Processes resume exactly where they stopped
Costs	Running processes are lost; builds, servers and terminals must be redone; a restarted agent may take a different path	Snapshots are as large as the VM's RAM and slow to write

Why we choose disk + memory: coding agents pause while idle (waiting on a human, an email, or another agent), when no one is waiting. They resume when someone is. A slow pause costs little; a slow or lossy resume costs a lot.

How we solve it, in two layers

Layer 1: Data plane (what we wrap)
Saves and restores one sandbox. Proven in our Firecracker spike.

- **Capture:** save memory and disk together as one snapshot, so a restore never pairs memory with a mismatched disk. Built on Firecracker microVM snapshots.
- **Store:** write the snapshot to the node's disk first, then upload it to GCS in the background.
- **Restore:** load the snapshot into a new Firecracker process, with its own network so two restored copies never conflict. *(Firecracker's "Network Connectivity for Clones" guide)*

Layer 2: Control plane (what we build)
Decides when to pause and keeps clients connected. This is where our design differs from E2B Embed: it pauses on a timeout, and we pause on the agent's state.

- **Decide when to pause:** based on what the agent is doing, not on a timer.

Step	What happens
Collect signals	Agent hooks, process activity, network activity, leases
Classify	Is the agent working, waiting, or idle?
Decide	Pause only if waiting or idle. Resume on the next request.

  Safety rules: a grace period before pausing; leases that expire, so a crashed agent can't keep a sandbox awake; a cap on how long a sandbox stays awake. Precedent: Daytona and Blaxel pause based on activity, AgentENV uses leases, and Kubernetes autoscalers wait out a stabilization window before scaling down.
- **Reconnect:** clients reconnect to the agent's event stream and receive only what they missed while it was paused. Precedent: Runloop's Axon + Broker, an ordered, saved event stream that survives interrupted sessions and replays history *(Runloop: Axons, Remote Agent SDK, May 5 2026; Axons, part 1)*. Daytona's Background Agents SDK shows the problem in practice: a developer saves session IDs, reconnects later, and fetches only unseen events *(Daytona: Background Agents SDK, Apr 24 2026; Claude Managed Agents)*.

Baseline we compare against: E2B, which pauses when a timeout expires and resumes on incoming traffic.

Current alternatives

Provider	What a pause saves	What triggers a pause	How it resumes	Open source / self-host	Built for
E2B Embed	Disk + memory by default; disk only optional	A timeout. "Pause on timeout" is opt-in; the default is kill	API call; auto-resume on traffic is opt-in	Apache-2.0, one self-hosted node	Self-hosting where data must stay put
AgentENV	Disk + memory, saved incrementally; pauses in under 100 ms (vendor figure). Snapshots go to S3-compatible storage or a shared filesystem	Idle sandboxes release CPU and memory; auto-pause on lease expiry (15 s default)	When new work arrives, in under 50 ms (vendor figure); E2B-compatible API	Self-hosted (Docker Compose multi-node, Kubernetes), Firecracker; MIT per AI Weekly	Agents needing isolated environments, including autonomous coding agents; RL training at scale
Daytona	Containers: disk only. VMs: disk + memory	Auto-stop after 15 min with no API activity; misses detached processes	API call	Repo is AGPL-3.0; check whether VM pause is in the open-source version	Dev environments and agents
Blaxel	Disk + memory; outside connections drop	Connection idleness: ~15 s with no connection, or 15 min of an idle open one	On reconnect, in under 25 ms (vendor figure)	Managed	Agent sandboxes
Modal	Disk + memory (alpha); snapshots expire after 7 days and taking one ends the sandbox	Manual only	New sandbox from the snapshot	Managed	Sandboxes for ML/agent workloads
Fly.io Sprites	Disk (checkpoints restart the environment)	30 s of inactivity	On demand, including an incoming HTTPS request	Managed	Persistent VMs for coding agents

NetClode	Disk only: the workspace volume (repo, tools, Docker data, agent session files) persists in S3; the VM is deleted, so running processes are lost	Manual pause, or automatically when more than 5 sessions are active (the oldest inactive one is paused)	A new VM mounts the saved volume; the conversation continues from session files plus event history in Redis Streams	Public on GitHub with an Ansible playbook for self-hosting; license not stated in the post	Personal, single-tenant remote coding agent (Kata Containers + Cloud Hypervisor)
Ours	Disk + memory, saved as one unit	Agent state: hooks + leases + measured activity, after a grace period	Next task or message; clients replay missed events	Open source, self-hosted, one node	Solo devs and small teams running long-lived coding agents

Where we sit - Use case

- We're building a narrow, open-source clone of E2B Embed's memory-persistent pause and resume, with one change: pausing is driven by what the agent is doing, not by a timeout or network traffic.
- Existing auto-pause/auto-stop is blind to agent state. Daytona's auto-stop misses detached processes, and timeouts cut off agents mid-turn (see Evidence). Our control plane pauses only when the agent is in a safe-to-pause state, and resumes on the next event.
- Use case: small teams or solo developers who self-host coding agents in full dev-environment sandboxes and want idle compute reclaimed without interrupting work in progress.
- Tradeoff: smarter pause decisions in exchange for coupling to agent signals (hooks, leases, process activity). Teams whose agents can't expose those signals are better served by E2B Embed's traffic-based pausing.

Model project and sources

- **Model project:** E2B Embed (what we're narrowly cloning). Firecracker: what we wrap. Daytona: closed-source reference for the container vs. VM tradeoff.
- **Sources:**
  - MicroVMs Support
    - https://www.zenml.io/llmops-database/designing-agent-sandbox-infrastructure-at-scale-from-runtime-to-orchestration
    - https://stanislas.blog/2026/02/netclode-self-hosted-cloud-coding-agent/ (microVMs)
    - https://pkg.go.dev/github.com/clawkwork/clawk@v0.1.0#section-readme (VM for macOS, Firecracker microVM for Linux)
    - https://browser-use.com/posts/two-ways-to-sandbox-agents
    - https://builders.cortex.io/blog/sandboxing-agents-part-1/

Open questions

- What is our MVP architecture for the control plane (SDK)?
- Prototype client app where the sandbox has a full stack app to better understand how our control plane (SDK) can plugin easily for the client.
- For idleness, which pattern do we use? What is our model solution?
  - Which agent signals are reliable enough to trust: hooks, leases, or process activity?

Stretch goals

- **Stable Psuedo Terminal**: a feature that allows the agent to interact with the sandbox, but what classifies one as stable or not? (a way for an agent to interact with the sandbox and it's state)
- **Observability:** each sandbox's state (running, idle, paused), pause/resume times and snapshot sizes.
  - **Replay**: documenting all the steps in the sandbox to allow for replay of those steps (Kitaru)
- **Concurrency / forking:** copying a running sandbox to try several approaches from one point. Supported for evals; still needs evidence for our use case.
- **Branching**: state persistence in terms of sandbox duplication
- **Two-tier pausing:** memory pause for short idle, filesystem-only for long idle.

Evidence

For needing memory persistence

Capy (which uses Daytona) talking about pause requirements for their agentic coding app

> With execution happening fully inside these sandboxes, they had to hold up under the demands of real engineering work. Developers routinely hand off long-running tasks to their agents, pause them mid-flight, and resume work days later after reviewing the output. Sandboxes had to persist for the duration of agent execution and restore from a precise checkpoint to avoid wasted compute, lost progress, and broken builds.

Reddit post complaining about losing processes when a sandbox dies

How are you all handling state for long-running agents? Stateless sandboxes are eating my evenings

by u/MaleficentWedding545 · 3 points · 14 comments · Posted 166 days ago

ok I want to know if I am the only one. been running a local coding agent against qwen3 coder on a 4090 box, with a remote sandbox for the actual code execution. every time the sandbox dies (idle timeout, host restart, whatever) I lose the entire working directory, installed deps, any process state the agent built up.

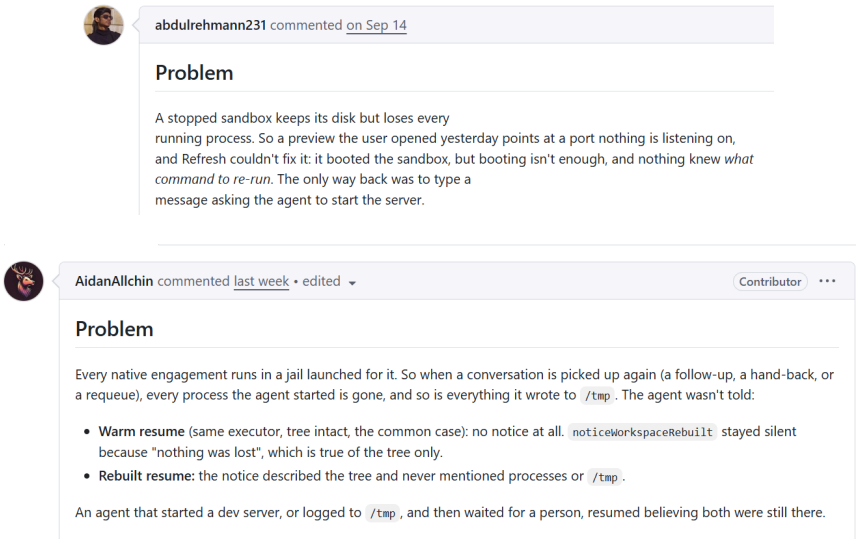
it is not just annoying, it costs real time. timed one resume cycle last night for a project the agent had been iterating on for two weeks. pip install of the repo deps 33s. model warmup and context reload 38s. restoring the working dir from s3 because I had to write my own checkpoint layer 17s. plus a few seconds of orchestration glue. total 91s before the agent can take its next turn. on a fresh session this is fine. on the 14th resume of a long-running project it makes me want to throw the machine out a window.

the obvious mental model is treat the sandbox as a persistent unix box and never let it die. but every provider I looked at has some flavor of timeout. e2b paused sandboxes get deleted after 30 days and pause takes about 4s per gb of ram. modal memory snapshots expire after 7 days and are still alpha. daytona archives at 30. fly machines stop is closer to what I actually want but the cold start tax shows up again on resume. blaxel.ai claims infinite standby with sub 25ms resume but I have not stress tested it past a week yet.

is anyone actually solving this without building your own checkpoint layer on top of s3 and a state machine. what is your setup. running everything in one persistent vm and eating the idle cost. snapshotting filesystem only and accepting that processes get nuked. something with temporal as the durable execution layer wrapping a sandbox provider underneath.

curious especially what the loca LLM folks are doing because cold-loading a 32b quant on every sandbox resume is brutal.

GitHub PRs complaining about losing running processes when sandbox stops



abdulrehmann231 commented on Sep 14

Problem

A stopped sandbox keeps its disk but loses every
running process. So a preview the user opened yesterday points at a port nothing is listening on,
and Refresh couldn't fix it: it booted the sandbox, but booting isn't enough, and nothing knew *what command to re-run*. The only way back was to type a
message asking the agent to start the server.

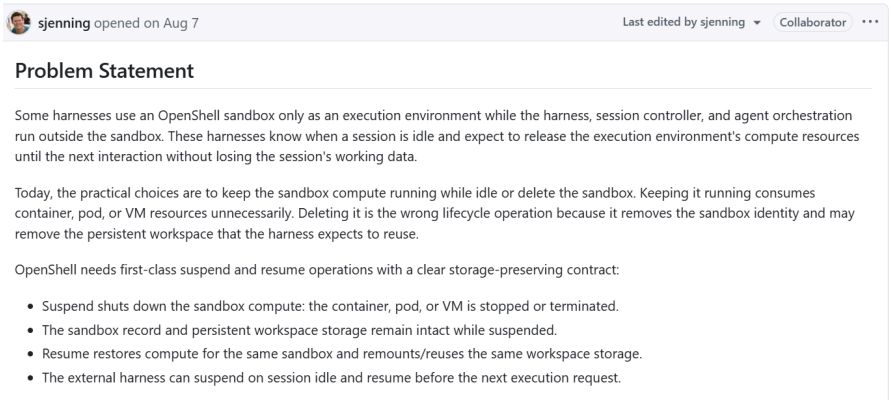
AidanAllchin commented last week • edited — Contributor

Problem

Every native engagement runs in a jail launched for it. So when a conversation is picked up again (a follow-up, a hand-back, or a requeue), every process the agent started is gone, and so is everything it wrote to `/tmp`. The agent wasn't told:

- **Warm resume** (same executor, tree intact, the common case): no notice at all. `noticeWorkspaceRebuilt` stayed silent because "nothing was lost", which is true of the tree only.
- **Rebuilt resume:** the notice described the tree and never mentioned processes or `/tmp`.

An agent that started a dev server, or logged to `/tmp`, and then waited for a person, resumed believing both were still there.



sjenning opened on Aug 7 — Last edited by sjenning — Collaborator

Problem Statement

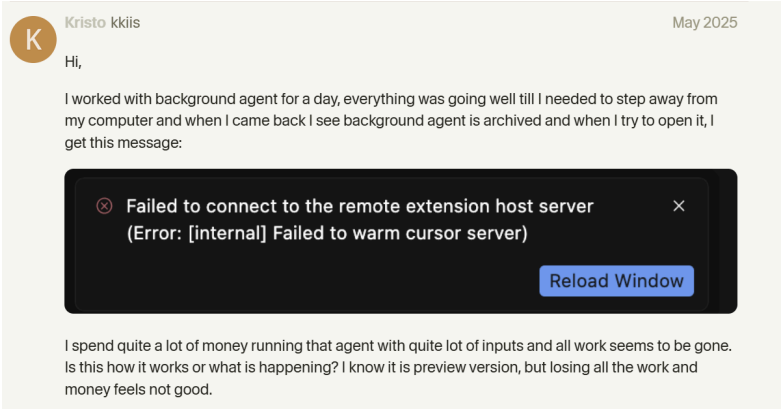
Some harnesses use an OpenShell sandbox only as an execution environment while the harness, session controller, and agent orchestration run outside the sandbox. These harnesses know when a session is idle and expect to release the execution environment's compute resources until the next interaction without losing the session's working data.

Today, the practical choices are to keep the sandbox compute running while idle or delete the sandbox. Keeping it running consumes container, pod, or VM resources unnecessarily. Deleting it is the wrong lifecycle operation because it removes the sandbox identity and may remove the persistent workspace that the harness expects to reuse.

OpenShell needs first-class suspend and resume operations with a clear storage-preserving contract:

- Suspend shuts down the sandbox compute: the container, pod, or VM is stopped or terminated.
- The sandbox record and persistent workspace storage remain intact while suspended.
- Resume restores compute for the same sandbox and remounts/reuses the same workspace storage.
- The external harness can suspend on session idle and resume before the next execution request.

Cursor Forum user report losing RAM to be a serious problem when working with agentic coding apps such as Background Agent



Kristo kkiis — May 2025

Hi,

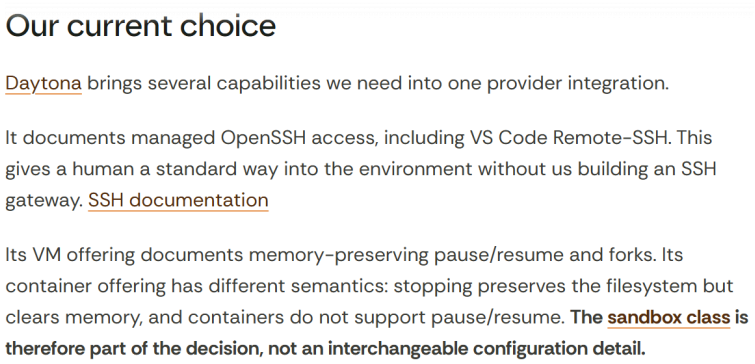
I worked with background agent for a day, everything was going well till I needed to step away from my computer and when I came back I see background agent is archived and when I try to open it, I get this message:

Failed to connect to the remote extension host server
(Error: [internal] Failed to warm cursor server)

Reload Window

I spend quite a lot of money running that agent with quite lot of inputs and all work seems to be gone. Is this how it works or what is happening? I know it is preview version, but losing all the work and money feels not good.

ZenML labs blog emphasizing memory preserving capabilities of Daytona



Our current choice

Daytona brings several capabilities we need into one provider integration.

It documents managed OpenSSH access, including VS Code Remote-SSH. This gives a human a standard way into the environment without us building an SSH gateway. SSH documentation

Its VM offering documents memory-preserving pause/resume and forks. Its container offering has different semantics: stopping preserves the filesystem but clears memory, and containers do not support pause/resume. **The sandbox class is therefore part of the decision, not an interchangeable configuration detail.**

For firecracker and narrower E2B

https://reddit.sentinel-team.org/posts/1t650iy/snapshots/2026-05-08T19%3A17%3A52.632086Z

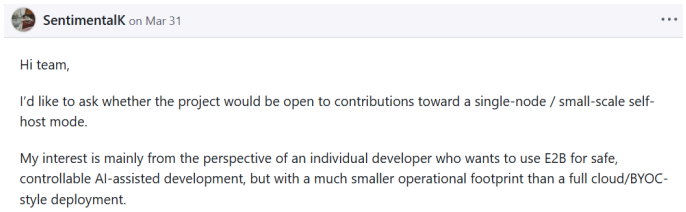
User found firecracker as the best sandbox approach for long-running agents



i ran AI agents on 5 sandbox setups for 6 weeks. firecracker won.

by u/AccomplishedFix3476 — 2 points — 4 comments — Posted 182 days ago

Small team wanting a narrower E2B with a smaller operational footprint



SentimentalK on Mar 31

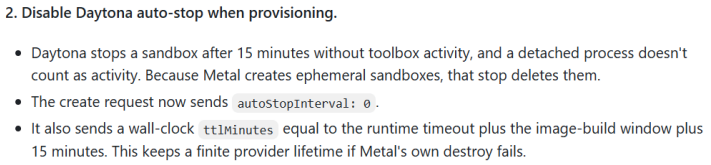
Hi team,

I'd like to ask whether the project would be open to contributions toward a single-node / small-scale self-host mode.

My interest is mainly from the perspective of an individual developer who wants to use E2B for safe, controllable AI-assisted development, but with a much smaller operational footprint than a full cloud/BYOC-style deployment.

For Daytona/E2B's idleness problems

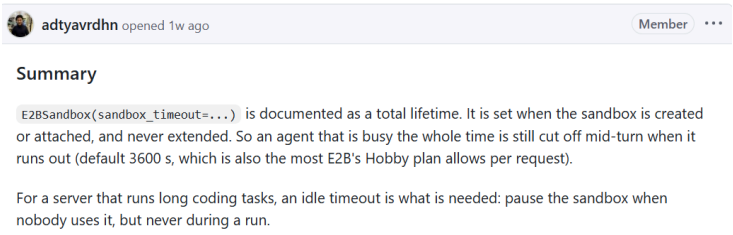
Github PR suggests disabling Daytona's auto-stop because it does not detect detached processes



2. Disable Daytona auto-stop when provisioning.

- Daytona stops a sandbox after 15 minutes without toolbox activity, and a detached process doesn't count as activity. Because Metal creates ephemeral sandboxes, that stop deletes them.
- The create request now sends `autoStopInterval: 0`.
- It also sends a wall-clock `ttlMinutes` equal to the runtime timeout plus the image-build window plus 15 minutes. This keeps a finite provider lifetime if Metal's own destroy fails.

Github PR pointing out that "agent that is busy the whole time is still cut off mid-turn" when sandbox timeout runs out



adtyavrdhn opened 1w ago Member

Summary

`E2BSandbox(sandbox_timeout=...)` is documented as a total lifetime. It is set when the sandbox is created or attached, and never extended. So an agent that is busy the whole time is still cut off mid-turn when it runs out (default 3600 s, which is also the most E2B's Hobby plan allows per request).

For a server that runs long coding tasks, an idle timeout is what is needed: pause the sandbox when nobody uses it, but never during a run.